SNES Assembly Adventure 05: Subroutines

21 Dec 2018

Update February 2022: All code examples from all articles in this series can now be found on Github in one repository

Welcome back, Adventurer. In this article, I'll introduce you to subroutines. You will learn

- What the stack is and how it works
- What subroutines are and how they work

We'll write a simple subroutine that adds the value in register X to the value in register Y.



I forgot to animate this...it's been a while

But before we dive deeper into subroutines, we first need to introduce the *stack*. I'll probably come back to the stack several times in future articles since it's such a great and important concept to understand. So this is by no way a complete description of the stack and all its functionality.

The Stack

The stack formally called a LIFO (last-in, first-out) structure, is a certain range of memory locations where we can store data. The most important characteristic of a stack is that it is a *chronological* structure. So whatever data we *pushed* last onto the stack, is the first data that gets *pulled* from the stack.

When we want to place data on the stack, we say we're **pushing** onto (the top) of the stack; when we want to retrieve that data again from the stack, we say we're **pulling** data from (the top of) the stack (*Note*: Some processor architectures use *pop* instead of *pull*; popping and pulling from stack are the same thing, just different words).

Imagine a stack of plates: Whenever we *push* a plate on top of the stack, we first need to *pull* the last plate we put on top of the stack before we can *pull* any of the plates beneath it.

Here is how the stack on the 65816 works. I'll first describe the operations in general, then I'll show you some code examples.

Whenever **we push data onto the stack**, the 65816 takes the value inside the internal *stack pointer* or SP and decrements it by either one, two, or three (we will see shortly why). Then it uses the new stack pointer (+1) as the address to store the data in memory. I'll explain the '(+1)' in a moment.

Whenever **we pull data from the stack**, the 65816 loads the data from the address stored in the stack pointer (+1), then increments the stack pointer by either one, two, or three.

Here is what the '(+1)' part in the paragraphs above means: The stack pointer on a 65816 always points *one byte beyond* the data we last pushed onto the stack. For example, if the stack pointer currently holds the value `$1ffa` and we pull one byte from the stack, the data stored at address `$1ffb` (stack pointer + 1) will be pulled, and the stack pointer set to `$1ffb`.

Likewise, if we push two bytes onto the stack, the stack pointer will decrement by two from `$1ffa` to `$1ff8`, the two bytes we push onto the stack are then stored at the addresses `$1ff9` and `$1ffa` respectively.

Let’s make this a bit clearer with a simple code example. This code will also include code to switch the 65816 from emulation to native mode and set the index registers X and Y to 16-bit, while the accumulator will operate in 8-bit mode.